INT161 · Basic Backend Development

Exam Walkthrough

Start the exam from the lecturer's template, step by step: what is a stub, what is broken, and the order to build it in.

Open the Project | Database in VS Code | Offline Prep | Tour of the Template
Config and First Boot | Build Order | Test Without Postman | Error Middleware
Time Plan and Checklist | Glossary

01

Open the Project

The lecturer gives you a **project template** next to your spec. This class walks it from the first click to the zip. The person in every example, Somchai Jaidee, ID 67000000000, key AB12cd, is made up.

> ⚠ **Swap in yours**
> Wherever you see Somchai Jaidee, 67000000000 or AB12cd, use the name, ID and secret key from your own spec. Three places: the **header** comment on the first line of every file ({student-id} - {student-name}), STUDENT_ID in app.js, and name and secretKey in the student-info **route**.

1. Unzip the template. Check that you do not end up with a folder inside a folder of the same name.
2. Open the inner folder, the one holding package.json, as the project in your IDE.
3. Import the database from your SQL file. Section 2 shows how in VS Code.
4. Run **npm** install once, while the network is still on. Then turn the network off.
5. Open your spec in preview mode and read all of it before writing a line.

project folder

```
$ npm install
```

Ignore the empty file named npm and the .idea folder. They do nothing.

02

Database in VS Code

WebStorm has a database tool built in. VS Code does not, so import the SQL file another way. MySQL and Workbench are already installed from class 4. Use Workbench first, because it needs no command line.

Route 1: MySQL Workbench

1. Open your local **connection** and enter the root password.
2. File, Open SQL Script, pick init-database.sql.
3. Click the lightning button to run the whole script.
4. Refresh the Schemas panel. mobile_distributor appears with two tables.

While you are there, read the real table and column names. The page assumes brands and products, and your SQL file is the truth.

Route 2: command line

The command in the spec, mysql -u root -p < file.sql, does not work in PowerShell, which has no < operator. Pick one of these instead.

PowerShell

```
# run it through cmd, which understands <
> cmd /c "mysql -u root -p < database\init-database.sql"
# or pipe the file in
> Get-Content database\init-database.sql | mysql -u root -p
# or log in, then source it with forward slashes
> mysql -u root -p
mysql> source C:/Users/you/exam/database/init-database.sql
```

If Windows says mysql is not recognized, add the MySQL bin folder to **PATH**, for example C:\Program Files\MySQL\MySQL **Server** 8.0\bin. The version number in the path may differ on your machine. Open a new terminal afterwards.

Route 3: inside VS Code, optional

An extension such as SQLTools with its MySQL **driver** lets you run queries in the editor. It is a nice extra, not a need. You must install it before the exam, because the exam is offline.

Check it worked

```mysql
mysql> USE mobile_distributor; SHOW TABLES;
mysql> SELECT COUNT(*) FROM products;
```

> ⚠ **Service must be running**
> If the connection is refused, MySQL is not running. Press Win+R, type services.msc, find MySQL80 (the name varies by install) and start it. A wrong password later in db-config.js gives ER_ACCESS_DENIED_ERROR.

03

Offline Prep

The exam is offline and you may run npm install once. Anything you need from the internet has to be in place before the network goes off. Run this list the day before, then again at the start of the exam.

1. Run **node** -v. **Express** 5 needs Node 18 or newer.
2. Install the VS Code extensions you plan to use: **REST Client**, and optionally SQLTools. They cannot be installed offline.
3. File, Open Folder, and pick the folder that holds package.json. The terminal then starts in the right place, and npm does not complain it cannot find package.json.
4. Run npm install once, with the network on. Check that node_modules appears.

5. Turn on File, Auto Save. A file you forgot to save is the classic reason a fix seems to do nothing.
6. Confirm Workbench still connects, and that the SQL file imports cleanly.
7. Turn the network off and do a full run on the template, from section 4 to the zip. Turning it back on during the exam counts as cheating.

04

Tour of the Template

The folder names end in $id. Leave them. Every import in the template already uses them, so renaming adds a chance to break something.

`app.js`

Starts the server. Mounts one router at "/". Has an empty error handler. You rewrite it.

`configs-$id/db-config.js`

The MySQL pool. Every value is a placeholder.

`routes-$id/route.js`

One generic router that returns placeholder text. Copy it per resource.

`services-$id/service.js`

Calls the repository and the mapper. Has bugs with null.

`repositories-$id/repository.js`

Generic SQL with table-name placeholders. Never returns 404 on update or delete.

`mappers-$id/`

api-response.js wraps the JSON. entity-dto-mapper.js is written for another database.

middlewares-$id/pagination-middleware.js

Reads page and limit. The offset math is wrong.

test-service.js

A scratch file to try the service without HTTP. Its import path is broken.

The template is a skeleton. Nothing in it answers the spec yet. You copy the generic route, service and **repository** once per **resource**, as in class 6: brand-route.js, brand-repository.js, product-route.js and so on. Keep the originals untouched as your reference.

05

Config and First Boot

Three small edits, then you boot. First, package.json has no start script, but the spec tells you to run npm start. Add it.

package.json

```
"scripts": {
  "start": "node src/app.js"
},
```

Second, fill in the **pool** in db-config.js. The database is mobile_distributor.

configs-$id/db-config.js

```
// 67000000000 - Somchai Jaidee
import mysql from 'mysql2/promise'
const db = mysql.createPool({
    host: 'localhost',
    user: 'root',
    password: 'your-mysql-password',
    database: 'mobile_distributor',
    dateStrings: ['DATE'], // release_date stays '2023-09-22', no timezone shift
})
export default db
```

Third, rewrite app.js. This also builds the identity **endpoint**, your first 10 points. The error handler goes before listen, after the **routers**.

src/app.js

```
// 67000000000 - Somchai Jaidee
import express from 'express'
import brandRouter from './routes-$id/brand-route.js'
import productRouter from './routes-$id/product-route.js'
import { errorHandler } from './middlewares-$id/error-middleware.js'
const STUDENT_ID = '67000000000'
const app = express()
app.use(express.json())
app.get(`/api/${STUDENT_ID}/student-info`, (req, res) => {
    res.status(200).json({
        success: true,
        data: { studentId: STUDENT_ID, name: 'Somchai Jaidee', secretKey: 'AB12cd' },
        timestamp: new Date().toISOString(),
    })
})
app.use(`/api/${STUDENT_ID}/brands`, brandRouter)
app.use(`/api/${STUDENT_ID}/products`, productRouter)
app.use(errorHandler) // after the routers, before listen
app.listen(3000, () => console.log('listening on port 3000'))
```

The imports point at files you create in the next sections. Make empty versions now, or comment those lines until you reach them.

Create the error handler first, because app.js imports it.

middlewares-$id/error-middleware.js

```
// 67000000000 - Somchai Jaidee
export class AppError extends Error {
    constructor(statusCode, message) {
        super(message)
        this.statusCode = statusCode
    }
}
export const errorHandler = (err, req, res, next) => {
    const status = err.code === 'ER_DUP_ENTRY' ? 409 : err.statusCode || 500
    res.status(status).json({
        success: false,
        error: { message: err.message },
        timestamp: new Date().toISOString(),
    })
}
```

boot and test

```
$ npm start
# second terminal, or the browser
$ curl localhost:3000/api/67000000000/student-info
```

> **Stop and check**
>
> You should see success true, your ID, your name and your key. If you do, question one is done. Commit to this order for the rest of the exam: finish one endpoint, test it, move on.

06

Build Order

Build in this order. Small and certain first, the biggest block of points once the pattern is warm.

1. student-info

app.js
10 pts. 200 with your name and key. Takes five minutes.

2. Brands

brand-repository, brand-route
GET list and GET one. 404 when missing.

3. Products list

pagination, repository, mapper, service, route
30 pts, the biggest. Summary DTO and the pagination block.

4. Product detail

repository (JOIN), mapper
200 or 404. brandName comes from the join.

5. POST

repository, route
201 saved, 409 when the SKU exists.

6. PUT and DELETE

repository, route
PUT 200, DELETE 204 with no body, 404 for both when missing.

Step 2: Brands

Copy repository.js to brand-repository.js and route.js to brand-route.js. Brands need no service or **mapper**: the table columns already match the **response**, so the route calls the

repository directly. This is a shortcut that works for a list this small.

repositories-$id/brand-repository.js

```
// 67000000000 - Somchai Jaidee
import db from '../configs-$id/db-config.js'
const FIND_ALL_SQL = 'SELECT * FROM brands'
const FIND_ONE_SQL = 'SELECT * FROM brands WHERE id = ?'
export async function findAll() {
    const [rows] = await db.query(FIND_ALL_SQL)
    return rows
}
export async function findOne(id) {
    const [rows] = await db.query(FIND_ONE_SQL, [id])
    return rows[0] ?? null // null, not a throw: the route decides it is a 404
}
```

routes-$id/brand-route.js

```
// 67000000000 - Somchai Jaidee
import * as repo from '../repositories-$id/brand-repository.js'
import { apiResponse } from '../mappers-$id/api-response.js'
import { AppError } from '../middlewares-$id/error-middleware.js'
import express from 'express'
const router = express.Router()
router.get('/', async (req, res) => {
    await apiResponse(res, await repo.findAll())
})
router.get('/:id', async (req, res) => {
    const brand = await repo.findOne(req.params.id)
    if (!brand) throw new AppError(404, `Brand ${req.params.id} not found`)
    await apiResponse(res, brand)
})
export default router
```

The template route does not return the **timestamp** the spec asks for. Fix that once, in api-response.js, and every endpoint gets it.

mappers-$id/api-response.js

```
// in both success() and paginate(), add one line after `data`:
        timestamp: new Date().toISOString(),
```

Test: GET /brands, GET /brands/1, GET /brands/99999. The last must be 404.

Step 3: Products list

Three template files are wrong for this step. The **pagination middleware** multiplies by the raw **page size** string, and returns null when page is missing, but the spec defaults to page 1. Replace it.

middlewares-$id/pagination-middleware.js

```
// 67000000000 - Somchai Jaidee
export const queryParser = () => (req, res, next) => {
    const page = Math.max(parseInt(req.query.page) || 1, 1)
    const limit = Math.min(Math.max(parseInt(req.query.limit) || 10, 1), 100)
    req.pagination = { page, limit, offset: (page - 1) * limit } // times limit, not the raw stri
    next()
}
```

The mapper was written for another database (productCode, MSRP). Rewrite it for your columns. Detail extends **summary**, so the fields live in one place.

mappers-$id/entity-dto-mapper.js

```
// 67000000000 - Somchai Jaidee
export default class EntityDtoMapper {
    static toSummaryDTO(e) {
        return {
            id: e.id, sku: e.sku, name: e.name, brandId: e.brand_id,
            category: e.category, price: Number(e.price), status: e.status,
        }
    }
    static toDetailDTO(e) {
        return {
            ...this.toSummaryDTO(e),
            brandName: e.brand_name, ramGb: e.ram_gb, storageGb: e.storage_gb,
            color: e.color, stockQuantity: e.stock_quantity, releaseDate: e.release_date,
        }
    }
    static map(entity, viewType) {
        return viewType == 'summary' ? this.toSummaryDTO(entity) : this.toDetailDTO(entity)
    }
    static maps(entities, viewType) {
        const { totalItems } = entities
        const out = entities.map(item => this.map(item, viewType))
        out.totalItems = totalItems // apiResponse reads the total from here
        return out
    }
}
```

The repository must also count the rows, or the response cannot say totalPages. The template defines a count **query** and never runs it. This file holds the detail join, the SKU check and the 404 logic for the next steps too.

repositories-$id/product-repository.js

```
// 67000000000 - Somchai Jaidee
import db from '../configs-$id/db-config.js'
const FIND_ALL_SQL = 'SELECT * FROM products LIMIT ? OFFSET ?'
const COUNT_SQL = 'SELECT COUNT(*) AS total FROM products'
const FIND_ONE_SQL = `SELECT p.*, b.name AS brand_name
    FROM products p JOIN brands b ON b.id = p.brand_id WHERE p.id = ?`
const FIND_SKU_SQL = 'SELECT id FROM products WHERE sku = ?'
const CREATE_SQL = 'INSERT INTO products SET ?'
const UPDATE_SQL = 'UPDATE products SET ? WHERE id = ?'
const REMOVE_SQL = 'DELETE FROM products WHERE id = ?'
export async function findAll({ limit, offset }) {
    const [rows] = await db.query(FIND_ALL_SQL, [limit, offset])
    const [[{ total }]] = await db.query(COUNT_SQL)
    rows.totalItems = total
    return rows
}
export async function findOne(id) {
    const [rows] = await db.query(FIND_ONE_SQL, [id])
    return rows[0] ?? null
}
export async function create(data) {
    const [dup] = await db.query(FIND_SKU_SQL, [data.sku])
    if (dup.length) return null // SKU already exists, the route answers 409
    const [result] = await db.query(CREATE_SQL, [data])
    return findOne(result.insertId)
}
export async function update(id, data) {
    const [result] = await db.query(UPDATE_SQL, [data, id])
    return result.affectedRows ? findOne(id) : null
}
export async function remove(id) {
    const [result] = await db.query(REMOVE_SQL, [id])
    return result.affectedRows > 0
}
```

services-$id/product-service.js

```
// 67000000000 - Somchai Jaidee
import * as repo from '../repositories-$id/product-repository.js'
import mapper from '../mappers-$id/entity-dto-mapper.js'
const detail = (row) => row && mapper.map(row, 'full') // null stays null
export async function getAllObjects(pagination) {
    return mapper.maps(await repo.findAll(pagination), 'summary')
}
export async function getObject(id) { return detail(await repo.findOne(id)) }
export async function createObject(data) { return detail(await repo.create(data)) }
export async function updateObject(id, data) { return detail(await repo.update(id, data)) }
export async function removeObject(id) { return repo.remove(id) }
```

routes-$id/product-route.js

```
// 67000000000 - Somchai Jaidee
import * as service from '../services-$id/product-service.js'
import { apiResponse } from '../mappers-$id/api-response.js'
import { queryParser } from '../middlewares-$id/pagination-middleware.js'
import { AppError } from '../middlewares-$id/error-middleware.js'
import httpStatus from 'http-status'
import express from 'express'
const router = express.Router()
const notFound = (id) => new AppError(404, `Product ${id} not found`)
router.get('/', queryParser(), async (req, res) => {
    const data = await service.getAllObjects(req.pagination)
    await apiResponse(res, data, req.pagination)
})
router.get('/:id', async (req, res) => {
    const data = await service.getObject(req.params.id)
    if (!data) throw notFound(req.params.id)
    await apiResponse(res, data)
})
router.post('/', async (req, res) => {
    const data = await service.createObject(req.body)
    if (!data) throw new AppError(409, 'SKU already exists')
    await apiResponse(res, data, null, httpStatus.CREATED)
})
router.put('/:id', async (req, res) => {
    const data = await service.updateObject(req.params.id, req.body)
    if (!data) throw notFound(req.params.id)
    await apiResponse(res, data)
})
router.delete('/:id', async (req, res) => {
    if (!await service.removeObject(req.params.id)) throw notFound(req.params.id)
    res.status(httpStatus.NO_CONTENT).end()
})
export default router
```

Test the list with page=1&**limit**=10, then page=2. Compare total, totalPages and hasNextPage against the spec example.

Steps 4 to 6: Detail, POST, PUT, DELETE

The files above already cover them, so these steps are mostly testing. Check each against its status:

- GET /products/1 returns 200 with brandName. GET /products/99999 returns 404.
- POST the **body** from your spec. 201 the first time, 409 the second time with the same sku. The body is **snake_case** and so are the columns, so it goes into the INSERT as is.
- PUT /products/:id with a changed field returns 200. A missing id returns 404.
- DELETE returns 204 with an empty body. A second DELETE of the same id returns 404.

To try a layer without **HTTP**, rewrite test-service.js. The template imports office-service.js, a file that does not exist.

src/test-service.js

```
import * as service from './services-$id/product-service.js'
console.log(await service.getObject(1))
console.log(await service.getObject(99999)) // null, no crash
process.exit() // the pool keeps Node alive otherwise
```

> ⚠ **Assumed names**
>
> Your SQL file was not available when this page was written. The table names brands and products and the column names come from the response fields in the spec. If your IDE shows different names, change the SQL strings to match.

07

Test Without Postman

A **browser** only sends GET. You need POST, PUT and DELETE too, and you will not have **Postman**. The REST Client extension (id humao.rest-client) turns a plain text file into clickable **requests**. Install it before the exam.

Create test.http in the project root. A Send Request link appears above every request. Click it and the response opens beside the file. Lines starting with ### separate requests.

test.http

```
@base = http://localhost:3000/api/67000000000
### student-info: 200
GET {{base}}/student-info
### brands: 200
GET {{base}}/brands
### brand 99999: 404
GET {{base}}/brands/99999
### products page 1: 200 with pagination
GET {{base}}/products?page=1&limit=10
### product detail: 200, then 404 for id 99999
GET {{base}}/products/1
### create: 201 the first time, 409 the second
POST {{base}}/products
Content-Type: application/json
{ "sku": "TEST-SKU-999", "name": "Test Phone 2026", "brand_id": 1, "category": "FLAGSHIP",
  "price": 29900.00, "ram_gb": 8, "storage_gb": 256, "color": "Black",
  "stock_quantity": 10, "status": "AVAILABLE", "release_date": "2026-01-01" }
### update: 200, 404 for a missing id
PUT {{base}}/products/1
Content-Type: application/json
{ "price": 31900.00 }
### delete: 204, 404 the second time
DELETE {{base}}/products/76
```

- Replace the ID in @base with yours. The product id 76 in the DELETE is an example: delete the product your POST created, using the id the POST returned.
- Each request names the status you should see. Compare it with the status line at the top of the response.
- No extension? Use **curl**.exe. In Windows PowerShell 5, curl alone is an alias for another command, so type the .exe. Quoting **JSON** on the command line is painful, so prefer the .http file.
- test.http is yours. Leave it out of the zip or keep it, it does not matter for grading.

08

Error Middleware

The template ends app.js with an empty handler, app.use((err, req, res, next) => {}). It catches every error and answers nothing, so the request hangs until it times out. You already replaced it in section 5. Here is how the pieces connect.

- A route **throws** new AppError(404, message). Express 5 passes errors from **async** routes to the handler by itself, so no **try and catch** and no next(err).
- The handler reads statusCode from the error. A database duplicate-entry error (ER_DUP_ENTRY) becomes 409. Anything without a status becomes 500.
- The handler must be registered after every router. A handler registered before them never sees their errors.

> **Where each status comes from**
>
> 404 is thrown in the route when the repository returns null or false. 409 is thrown when create returns null, or comes from the database. 201 and 204 are set on the success path. Nothing else needs a status.

09

Time Plan and Checklist

Two hours is enough if you do not stall. A rough plan:

- 15 min: open, database, config, app.js, student-info.
- 15 min: brands.
- 30 min: products list. It carries the most points, so do not rush it.
- 35 min: detail, POST, PUT, DELETE.
- 25 min: run the **checker**, fix the failures, zip. Keep this slice. Never spend it on a hard endpoint.

When something fails, look here first:

`Server starts, request never answers`

An error was swallowed. Is app.use(errorHandler) in app.js? Is it before listen?

Cannot find module

A wrong import path. The folder names keep their $id suffix, so copy them exactly.

Table does not exist

The table names in this page (brands, products) are a guess. Read them in the IDE data source and match.

Price comes back as a string

MySQL sends DECIMAL as text. The mapper wraps it in Number().

Page 2 shows the wrong rows

The offset used the raw string. Use the pagination middleware from this page.

Before you zip, run through this:

1. The header comment of every file you touched has your own ID and name.
2. app.js has your STUDENT_ID, name and secret key. Every route sits under /api/your-id.
3. node tests/test-runner.mjs your-id passes every case it can.
4. Delete node_modules, zip only the work folder, name it your-id_exam.zip.

> **Rehearse**
> Do the whole thing once, offline, with a timer, before the real exam. The first run always costs you time on setup. The second does not.

10

Glossary

async
: Start a slow job, keep working, and get called back when it finishes, instead of waiting.

body
: The data a request carries after the headers. For an API it is usually JSON.

browser

The client most people use, such as Chrome, Edge or Safari.

checker

The script tests/test-runner.mjs. It calls your running server, checks each endpoint against the spec and prints a score.

Client

The side that asks. Usually a browser, but it can be any program or device.

connection

An open line between your program and the database. You need one before you can send any SQL.

curl

A command line client. It sends any HTTP method, so it can test POST and PUT. Add -i to see the response headers.

driver

A library that knows how to talk to one kind of database. For MySQL in Node it is mysql2.

endpoint

One method plus path an API answers, such as GET /api/subjects.

Express

A small web framework for Node.js. It adds routing and middleware on top of the built-in http module.

header

Extra information about a message, sent as name: value lines, such as Content-Type.

HTTP

HyperText Transfer Protocol: the plain-text language a browser and a web server use to talk. One side sends a request, the other sends a response.

JSON

JavaScript Object Notation: a plain text format for data that both humans and programs can read.

limit

How many records one page holds. The client sends it as limit, and the server caps it.

mapper

A class that turns an entity into a DTO, so the conversion lives in one reusable place.

middleware

A function (req, res, next) that runs between the request and the route handler. It can change the request, answer it, or pass it on.

node

A runtime that runs JavaScript outside the browser, so the same language can write the back end.

npm

Node Package Manager: installs libraries such as express into node_modules.

page size

How many records one page holds. The client sends it as limit, and the server caps it.

pagination

Returning a big list one page at a time, so responses stay small and fast.

PATH

A Windows setting that lists the folders searched when you type a command. If the MySQL bin folder is not in it, mysql is not recognized.

pool

A few ready connections kept open and shared. A request borrows one and gives it back, so the database is not flooded.

Postman

A tool for sending any HTTP method to your API, so you can test it before the front end exists.

project template

A saved starting project. A new project made from it already has the folders and the basic code.

query

The part of the URL after ?, like page=2&size=10. Used to filter or page a list.

repository

The layer that reads and writes the database. No business rules, no HTTP.

requests

What the client sends: method, URL, headers and sometimes a body.

resource

A thing the API gives access to, such as a product or a customer. In this course it maps to a database table.

response

What the server sends back: status code, headers and a body.

REST

A style for web APIs: resources named by URLs, actions chosen by HTTP method.

route

Matching an HTTP method and a URL path to the function that handles it.

routers

Matching an HTTP method and a URL path to the function that handles it.

Server

A program (or the machine running it) that waits for requests and answers them.

Service

The layer that holds the business rules. It calls repositories and knows nothing about HTTP.

snake_case

Words joined by underscores, like user_name. Common for database columns, not for URIs.

summary

Two sizes of DTO: summary has a few fields for lists, detail has everything for one item.

throws

Raise an error. Only Error objects can be thrown.

timestamp

A date and time value. A column like last_update can fill itself with the current time.

try and catch

try runs code that may fail. If it throws, execution jumps to catch, which receives the error.